Exercise 5 – Evaluation I

Introduction to Machine Learning

Hint: useful libraries

R

```
# Consider the following libraries for this exercise sheet:

library(ggplot2)
library(mlr3)
library(mlr3learners)
```

Python

```
# Consider the following libraries for this exercise sheet:

# general
import numpy as np
import pandas as pd
import math

# plots
import matplotlib.pyplot as plt

# sklearn
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
```

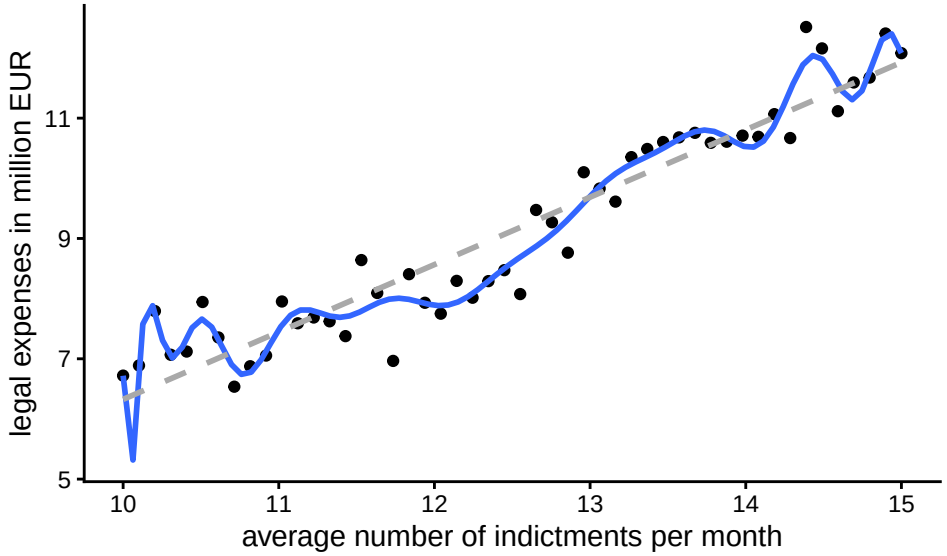
Exercise 1: Evaluating regression learners

Learning goals

1. Understand importance of using test error for evaluation
2. Discuss choice of performance metric in given situation

Imagine you work for a data science start-up and sell turn-key statistical models. Based on a set of training data, you develop a regression model to predict a customer's legal expenses from the average monthly number of indictments brought against their firm.

a) Due to the financial sensitivity of the situation, you opt for a very flexible learner that fits the customer's data ($n_{train} = 50$ observations) well, and end up with a degree-21 polynomial (blue, solid). Your colleague is skeptical and argues for a much simpler linear learner (gray, dashed). Which of the models will have a lower empirical risk if standard $L2$ loss is used?



Solution

Since the polynomial learner clearly achieves a better fit for the training data and some observations lie rather far from the regression line, which is strongly penalized by $L2$ loss, it will have lower empirical risk than the linear learner.

b) Why might evaluation based on training error not be a good idea here?

Solution

First and foremost, evaluation on the training data is almost never a good idea. Under certain conditions the training error does tell us something about generalization ability,

but for flexible learners and/or small training data it is deceptive due to optimistic bias. In this particular situation, we have few training observations and quite some points that look a little extreme. A low training error might be achieved by a learner that fits every quirk in the training data but generalizes poorly to unseen points with only slightly different distribution. Evaluation on separate test data is therefore non-negotiable.

c) Evaluate both learners using

   (i) mean squared error (MSE), and

   (ii) mean absolute error (MAE).

   First compute both measures **by hand** for the following small test set of five customers, where $y^{(i)}$ is the true legal expense and $\hat{y}^{(i)}_{\text{poly}}$, $\hat{y}^{(i)}_{\text{lin}}$ are the predictions of the polynomial and the linear learner:

Customer i	$y^{(i)}$	$\hat{y}^{(i)}_{\text{poly}}$	$\hat{y}^{(i)}_{\text{lin}}$
1	10	11	10
2	12	13	12
3	14	15	14
4	16	17	16
5	20	19	17

   State your performance assessment and explain potential differences. You can then verify the behaviour on the full test set using the code below.

R

```r
set.seed(123)
x_train <- seq(10, 15, length.out = 50)
y_train <- 10 + 3 * sin(0.15 * pi * x_train) + rnorm(length(x_train), sd = 0.5)
data_train <- data.frame(x = x_train, y = y_train)
set.seed(321)
x_test <- seq(10, 15, length.out = 10)
y_test <- 10 + 3 * sin(0.15 * pi * x_test) + rnorm(length(x_test), sd = 0.5)
data_test <- data.frame(x = x_test, y = y_test)
```

Python

```
np.random.seed(43)
x_train = np.linspace(10, 15, num=50)
y_train = 10 + 3 * np.sin(0.15 * math.pi * x_train)
y_train += np.random.normal(loc=0.0, scale=0.5, size=len(x_train))
data_train = pd.DataFrame({"y": y_train, "x": x_train})
np.random.seed(2238)
x_test = np.linspace(10, 15, num=50)
y_test = 10 + 3 * np.sin(0.15 * math.pi * x_test)
y_test += np.random.normal(loc=0.0, scale=0.5, size=len(x_test))
data_test = pd.DataFrame({"y": y_test, "x": x_test})
```

Solution

By hand. With residuals $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$:

i	$y^{(i)}$	$\hat{y}_{\text{poly}}$	r_{poly}	r^2_{poly}	$\hat{y}_{\text{lin}}$	r_{lin}	r^2_{lin}
1	10	11	-1	1	10	0	0
2	12	13	-1	1	12	0	0
3	14	15	-1	1	14	0	0
4	16	17	-1	1	16	0	0
5	20	19	$+1$	1	17	$+3$	9

Polynomial learner: $\text{MSE} = \frac{1}{5}(1+1+1+1+1) = 1$ and $\text{MAE} = \frac{1}{5}(1+1+1+1+1) = 1$.

Linear learner: $\text{MSE} = \frac{1}{5}(0+0+0+0+9) = 1.8$ and $\text{MAE} = \frac{1}{5}(0+0+0+0+3) = 0.6$.

So **MSE prefers the polynomial** learner ($1 < 1.8$), while **MAE prefers the linear** learner ($0.6 < 1$): the assessment depends on the chosen measure. The reason is the single extreme customer $i = 5$: the linear learner misses it by 3, which the MSE penalizes quadratically ($3^2 = 9$) so that this one point dominates its error, whereas the flexible polynomial fits it closely. The MAE weights the miss only linearly and therefore rewards the linear learner's otherwise perfect fit. The choice of performance metric thus encodes our requirements and can flip the decision.

On the full test set the same qualitative picture arises:

R

Data

```r
set.seed(123)
x_train <- seq(10, 15, length.out = 50)
y_train <- 10 + 3 * sin(0.15 * pi * x_train) + rnorm(length(x_train), sd = 0.5)
data_train <- data.frame(x = x_train, y = y_train)
set.seed(321)
x_test <- seq(10, 15, length.out = 10)
y_test <- 10 + 3 * sin(0.15 * pi * x_test) + rnorm(length(x_test), sd = 0.5)
data_test <- data.frame(x = x_test, y = y_test)
head(data_train)
```

```
         x        y
1 10.00000 6.719762
2 10.10204 6.888379
3 10.20408 7.793217
4 10.30612 7.066415
5 10.40816 7.119966
6 10.51020 7.943824
```

Train and evaluate

```r
# train learners
polynomial_learner <- lm(y ~ poly(x, 21), data_train)
linear_learner <- lm(y ~ x, data_train)

# predict with both learners
y_polynomial <- predict(polynomial_learner, data_test)
y_lm <- predict(linear_learner, data_test)

# compute errors
abs_differences <- lapply(
    list(y_polynomial, y_lm),
    function(i) abs(data_test$y - i)
)
errors_mse <- sapply(abs_differences, function(i) mean(i^2))
errors_mae <- sapply(abs_differences, mean)

sprintf(
    "MAE for Polynomial and Linear fit: %.4f, %.4f",
    errors_mae[1],
    errors_mae[2]
)
```

```
[1] "MAE for Polynomial and Linear fit: 0.4140, 0.3828"
```

```r
sprintf(
    "MSE for Polynomial and Linear fit: %.4f, %.4f",
    errors_mse[1],
    errors_mse[2]
)
```

```
[1] "MSE for Polynomial and Linear fit: 0.2731, 0.3032"
```

Python

Data

```
np.random.seed(43)
x_train = np.linspace(10, 15, num=50)
y_train = 10 + 3 * np.sin(0.15 * math.pi * x_train)
y_train += np.random.normal(loc=0.0, scale=0.5, size=len(x_train))
data_train = pd.DataFrame({"y": y_train, "x": x_train})
np.random.seed(2238)
x_test = np.linspace(10, 15, num=50)
y_test = 10 + 3 * np.sin(0.15 * math.pi * x_test)
y_test += np.random.normal(loc=0.0, scale=0.5, size=len(x_test))
data_test = pd.DataFrame({"y": y_test, "x": x_test})
print(data_train.head())
```

```
          y          x
0  7.128700  10.000000
1  6.549227  10.102041
2  6.824611  10.204082
3  6.763703  10.306122
4  7.484359  10.408163
```

Train and evaluate

```
# train polynomial regression
poly = PolynomialFeatures(degree=21)
poly_model = LinearRegression()
_ = poly_model.fit(poly.fit_transform(x_train.reshape(-1, 1)), y_train)

# train linear regression
lm_model = LinearRegression()
_ = lm_model.fit(
    x_train.reshape(-1, 1), y_train
)  # reshaping necessary because single feature

# make predictions
y_poly = poly_model.predict(poly.fit_transform(x_test.reshape(-1, 1)))
y_lm = lm_model.predict(x_test.reshape(-1, 1))

# compute errors
abs_differences = pd.DataFrame(np.column_stack([y_poly, y_lm])).apply(
    lambda y: abs(data_test.loc[:, "y"] - y)
)  # apply is componentwise
errors_mse = abs_differences.apply(lambda x: x**2).mean()  # ** is power symbol
errors_mae = abs_differences.mean()

print("MAE for Polynomial and Linear fit:", errors_mae, sep="\n")
```

```
MAE for Polynomial and Linear fit:
```

```
0    0.436762
1    0.421898
dtype: float64
```

```
print("MSE for Polynomial and Linear fit:", errors_mse, sep="\n")
```

```
MSE for Polynomial and Linear fit:
0    0.266721
1    0.276253
dtype: float64
```

Keep in mind, though, that this evaluation rests on a single holdout split, which is risky in general and particularly deceptive with so little data – for different test data the polynomial can easily end up worse on *both* measures. Take-home message: the choice of performance metric matters, and decisions based on a single train-test split are unreliable (which motivates Exercise 2).

Exercise 2: Importance of train-test split

Learning goals

1) Understand how strongly performance estimates can depend on data samples
2) Explain bias-variance trade-off of GE estimator for repeated sampling across different train/test splits

We consider the `CaliforniaHousing` data for which we would like to predict the median house value (`MedHouseVal`) from the median income in the neighborhood (`MedInc`).

R

Get data

```
# Adapt to version available on sklearn following description from
# https://gist.github.com/bgreenwell/b1330460eec5acf1c81fae71902e331c
# (figshare mirror of cal_housing.tgz -- same archive sklearn downloads)

setwd(tempdir())
url <- "https://ndownloader.figshare.com/files/5976036"
download.file(url, destfile = "cal.tar.gz", quiet = TRUE)
untar("cal.tar.gz")

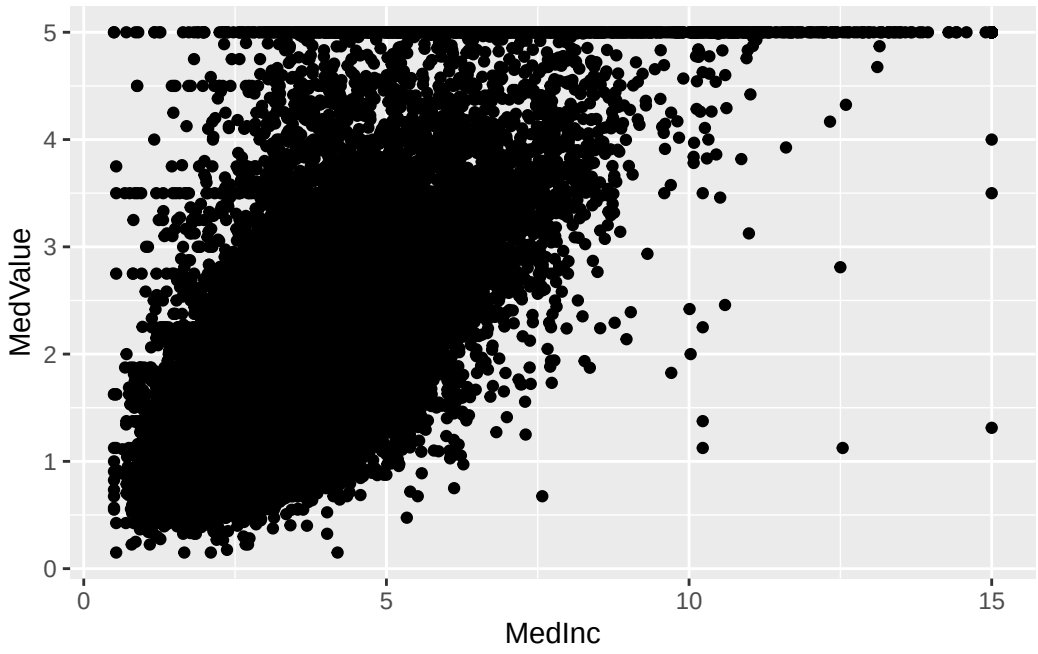
# Read the data into R and provide column names
```

```r
df_california <- read.csv("CaliforniaHousing//cal_housing.data", header = FALSE)
columns_index <- c(9, 8, 3, 4, 5, 6, 7, 2, 1)
df_california <- df_california[, columns_index]
names(df_california) <- c(
    "MedValue",
    "MedInc",
    "HouseAge",
    "AveRooms",
    "AveBedrms",
    "Population",
    "AveOccup",
    "Latitude",
    "Longitude"
)
df_california$MedValue <- df_california$MedValue / 100000
df_california <- df_california[, c("MedInc", "MedValue")]
head(df_california)
```

```
  MedInc MedValue
1 8.3252    4.526
2 8.3014    3.585
3 7.2574    3.521
4 5.6431    3.413
5 3.8462    3.422
6 4.0368    2.697
```

Inspect

```r
ggplot(df_california, aes(x = MedInc, y = MedValue)) +
    geom_point()
```



Python

Get data

```
from sklearn.datasets import fetch_california_housing

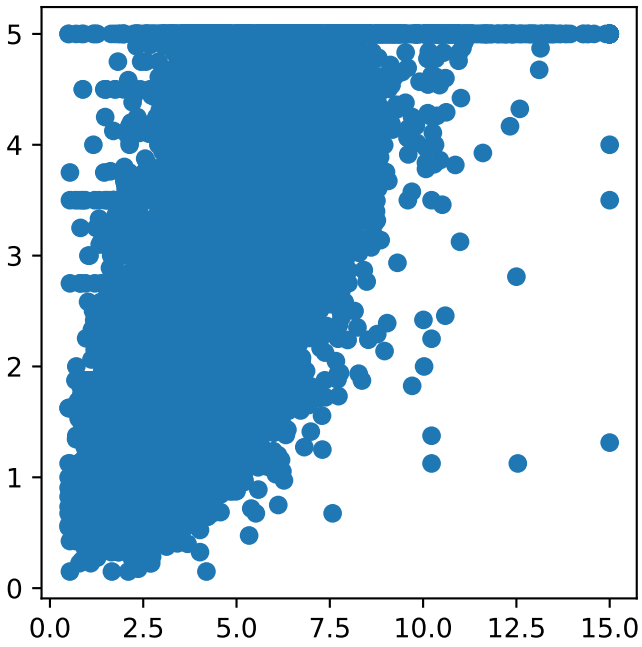
dataset_california = fetch_california_housing(as_frame=True)
df_california = dataset_california.frame.loc[:, ["MedInc", "MedHouseVal"]]
df_california.head()
```

```
   MedInc  MedHouseVal
0  8.3252        4.526
1  8.3014        3.585
2  7.2574        3.521
3  5.6431        3.413
4  3.8462        3.422
```

Inspect

```
# instantiate plot
fig = plt.figure(figsize=(4, 4))
# Creating axes instance
ax = fig.add_subplot()
# Creating plot
```

```
sp = ax.scatter(
    x=df_california.loc[:, ["MedInc"]], y=df_california.loc[:, ["MedHouseVal"]]
)
plt.show()
```



a) Use the first 100 observations as training data to compute a linear model and evaluate the performance of your learner on the remaining data using MSE.

Solution

Get the data, define a task and corresponding train-test split, and predict with trained model:

R

Get data

```
# (data is already loaded in `df_california` above)
head(df_california)
```

```
  MedInc MedValue
1 8.3252    4.526
2 8.3014    3.585
3 7.2574    3.521
4 5.6431    3.413
5 3.8462    3.422
6 4.0368    2.697
```

Perform split

```
df_california_sorted <- df_california[order(df_california$MedInc), ]
task <- TaskRegr$new(
    "housing",
    backend = df_california_sorted,
    target = "MedValue"
)
train_set = 1:100
test_set = setdiff(seq_len(task$nrow), train_set)
```

Train and evaluate

```
# train linear learner
learner <- lrn("regr.lm")
learner$train(task, row_ids = train_set)

# predict on test data
predictions <- learner$predict(task, row_ids = test_set)
sprintf("MSE of test data %.4f:", predictions$score())
```

```
[1] "MSE of test data 4.1522:"
```

Python

Get data

```
# (data is already loaded in `df_california` above)
df_california.head()
```

```
   MedInc  MedHouseVal
0  8.3252        4.526
1  8.3014        3.585
2  7.2574        3.521
```

```
3  5.6431        3.413
4  3.8462        3.422
```

Perform split

```
df_california_sorted = df_california.sort_values("MedInc")
df_california_sorted.reset_index(drop=True, inplace=True)
x_train = df_california_sorted.loc[:99, ["MedInc"]]
y_train = df_california_sorted.loc[:99, ["MedHouseVal"]]
x_test = df_california_sorted.loc[100:, ["MedInc"]]
y_test = df_california_sorted.loc[100:, ["MedHouseVal"]]
```

Train and evaluate

```
# train linear learner
lm_california = LinearRegression()
_ = lm_california.fit(X=x_train, y=y_train)

pred = lm_california.predict(x_test)
mse = mean_squared_error(y_test, pred)
print(f"MSE of test data: {mse:.4f}")
```

```
MSE of test data: 4.1522
```

b) What might be disadvantageous about the previous train-test split?

Solution

We have chosen the first few observations from a data set that is ordered by feature value, which is not a good idea because this is not a random sample and covers only a particular area of the feature space. Consequently, we obtain a pretty high test MSE (relatively speaking – we will see in the next exercise which error values we can usually expect for this task). Looking at the data, this gives us a regression line that does not reflect the overall data situation. Also, the training set is pretty small and will likely lead to poor generalization.

c) Now, sample your training observations from the data set at random. Use a share of 0.1 through 0.9, in 0.1 steps, of observations for training and repeat this procedure ten times. Afterwards, plot the resulting test errors (in terms of MSE) in a suitable manner.

Hint

R

`rsmp` is a convenient function for splitting data – you will want to choose the "holdout" strategy. Afterwards, `resample` can be used to repeatedly fit the learner.

Python

`from sklearn.model_selection import train_test_split` is a convenient function for splitting data. It has an optional parameter `random_state`, which can be used to split the data randomly in each iteration.

Solution

We repeat the above procedure for different train-test splits like so:

R

Train and predict for different splits

```
# define train-test splits
repetitions <- 1:10
split_ratios <- seq(0.1, 0.9, by = 0.1)

# create resampling objects with holdout strategy, using lapply for efficient computation
split_strategies <- lapply(split_ratios, function(i) rsmp("holdout", ratio = i))

# train linear learners and predict in one step (remember to set a seed)
set.seed(123)
results <- list()
lgr::get_logger("mlr3")$set_threshold("warn")
for (i in repetitions) {
    results[[i]] <- lapply(
        split_strategies,
        function(i) mlr3::resample(task, learner, i)
    )
}
```

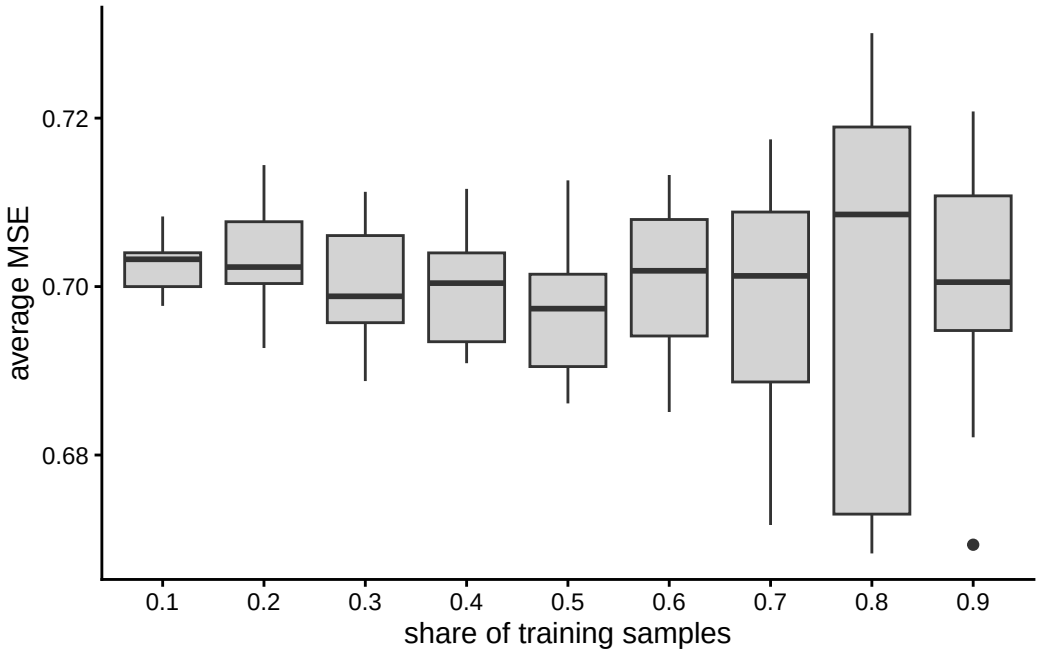
Compute errors

```r
errors <- lapply(
  repetitions,
  function(i) sapply(results[[i]], function(j) j$score()$regr.mse)
)

# assemble everything in data.frame and convert to long format for plotting
errors_df <- as.data.frame(do.call(cbind, errors))
errors_df$split_ratios <- split_ratios
errors_df_long <- reshape2::melt(errors_df, id.vars = "split_ratios")
names(errors_df_long)[2:3] <- c("repetition", "mse")
```

Visualize

```r
# plot errors vs split ratio
ggplot(errors_df_long, aes(x = as.factor(split_ratios), y = mse)) +
  geom_boxplot(fill = "lightgray") +
  theme_classic() +
  labs(x = "share of training samples", y = "average MSE")
```



Python

Train, predict and compute errors for different splits

```
x, y = df_california.loc[:, ["MedInc"]], df_california.loc[:, ["MedHouseVal"]]
split_ratio = np.linspace(0.1, 0.9, num=9)

result = pd.DataFrame()
for i in range(10):
    err = []
    for split in split_ratio:
        x_train, x_test, y_train, y_test = train_test_split(
            x, y, train_size=split, random_state=i + 10
        )
        lm_california.fit(X=x_train, y=y_train)
        pred = lm_california.predict(x_test)
        err.append(mean_squared_error(y_test, pred))
    result[i] = err
```

```
errors = result.transpose()
errors.columns = [f"0.{i}" for i in range(1, 10)]
```

Visualize

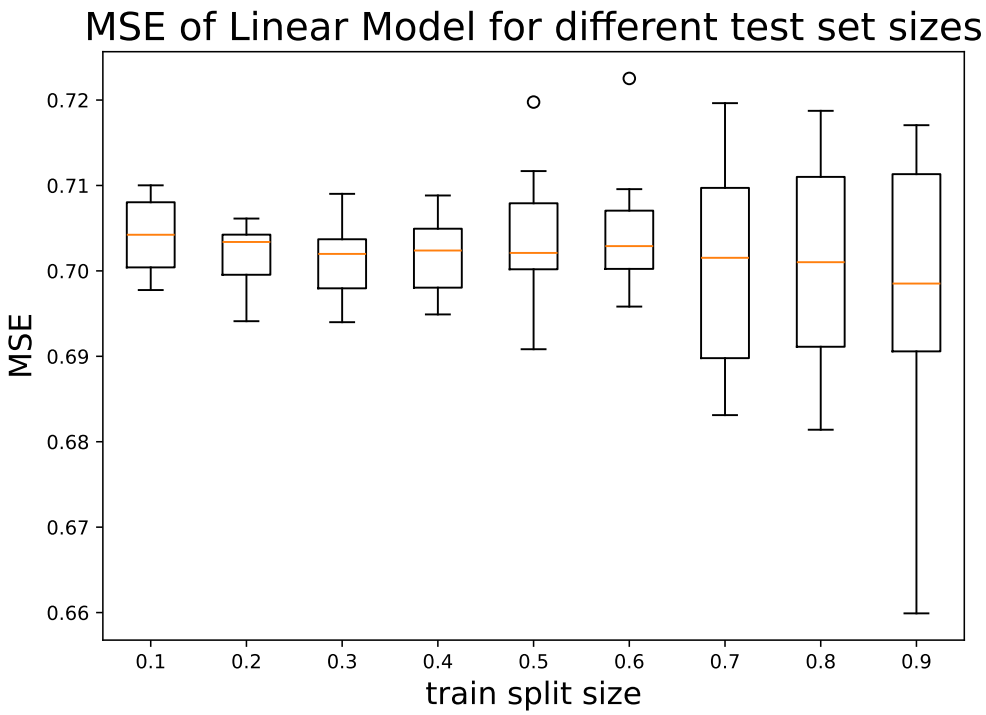
```
# plot your errors
# instantiate plot
fig = plt.figure(figsize=(8, 5.5))
# Creating axes instance
ax = fig.add_subplot()
# Creating plot
bp = ax.boxplot(errors)
# x-axis labels
ax.set_xticklabels([f"0.{i}" for i in range(1, 10)])
```

```
# Adding title
plt.title("MSE of Linear Model for different test set sizes", size=20)
```

```
plt.xlabel("train split size", size=16)
```

```
plt.ylabel("MSE", size=16)
```

```
# show plot
plt.show()
```



d) Interpret the findings from the previous question.

Solution

We can derive two conclusions:

i. A smaller training set tends to produce higher estimated generalization errors.

ii. A larger training set, at the expense of test set size, will cause high variance in the individual generalization error estimates.

Exercise 3: Choosing the right performance measure

The whole exercise is only for lecture group B!

Learning goals

Choose a suitable regression performance measure for a given situation

You work at an online shop and have trained a regression model that predicts the **weekly demand** (number of units sold) of each product in the catalog. Depending on what the

predictions are used for, a different performance measure is appropriate. For each of the following use cases of the *same* model, decide which measure from the lecture – **MSE**, **RMSE**, **MAE**, **MAPE**, **Spearman's** ρ, or R^2 – you would report, and justify your choice in one or two sentences.

a) To decide which products to feature on the homepage, you only need the products **ranked** correctly by predicted demand – the exact predicted numbers do not matter.

> **Solution**
>
> **Spearman's** ρ (the Spearman rank correlation coefficient). It is the correlation of the *ranks* of predictions and true values, so it rewards a model that orders the products correctly regardless of the absolute size of the errors – exactly what we need when only the ranking matters. (Being rank-based, it is also very robust to outliers.)

b) The catalog contains both niche products (a few units per week) and blockbusters (thousands per week). An absolute error of 10 units is negligible for a blockbuster but severe for a niche product, so you care about the **relative** error.

> **Solution**
>
> **MAPE** (mean absolute percentage error). It measures each residual relative to the true value, $\left|r^{(i)}/y^{(i)}\right|$, so a fixed absolute error counts more when the true demand is small (niche products) – exactly the relative view we want. (Caveat: it is undefined/unstable for $y^{(i)}$ near 0.)

c) You want to explain to your (non-technical) manager how much better the model is than simply always predicting each product's **average past demand**.

> **Solution**
>
> R^2 (the coefficient of determination). It relates the model's **sum of squared errors** (SSE) to that of the constant baseline that always predicts the mean demand: $R^2 = 0$ means "no better than predicting the mean", $R^2 = 1$ means "perfect fit", so it directly expresses the improvement over the trivial baseline.

Exercise 4: Properties of R^2

The whole exercise is only for lecture group A!

Learning goals

1. Interpret R^2 as a comparison to a featureless baseline model
2. Derive the range of R^2 on training data
3. Understand why R^2 can become negative on test data

Recall the coefficient of determination

$$R^2 = 1 - \frac{\text{SSE}_{\text{model}}}{\text{SSE}_{\text{const}}}, \qquad \text{SSE}_{\text{model}} = \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)}\right)^2, \qquad \text{SSE}_{\text{const}} = \sum_{i=1}^{n} \left(y^{(i)} - \bar{y}\right)^2,$$

where $\hat{y}^{(i)}$ are the model's predictions and $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y^{(i)}$.

a) The denominator $\text{SSE}_{\text{const}}$ is the SSE of a specific *featureless* baseline model. Which model is it, what does it predict, and in which sense does R^2 measure a model's performance *relative to* this baseline?

Solution

The baseline is the **featureless (constant) model** $f(\mathbf{x}) = \bar{y}$: it ignores all features and predicts the same value – the mean target $\bar{y}$ – for every observation. Its **sum of squared errors** (SSE) is exactly $\text{SSE}_{\text{const}}$.

Reading $R^2 = 1 - \frac{\text{SSE}_{\text{model}}}{\text{SSE}_{\text{const}}}$, the ratio $\text{SSE}_{\text{model}}/\text{SSE}_{\text{const}}$ sets the model's squared error *in relation to* the baseline's. So R^2 is not an absolute error measure but a **relative** one: it says how much of the baseline's error the model removes. Hence $R^2 = 0$ means the model is no better than simply predicting the mean $\bar{y}$, $R^2 = 1$ means a perfect fit ($\text{SSE}_{\text{model}} = 0$), and a value in between is the fraction by which the model reduces the baseline error. It helps to keep the two shares apart: the ratio $\text{SSE}_{\text{model}}/\text{SSE}_{\text{const}}$ is the fraction of the baseline error that **remains**, while $R^2 = 1 - \text{SSE}_{\text{model}}/\text{SSE}_{\text{const}}$ is the fraction that is **removed**. For example, $R^2 = 0.7$ means $\text{SSE}_{\text{model}}/\text{SSE}_{\text{const}} = 0.3$: the model's error is only 30% of the baseline's – it shrinks *to* 30% (equivalently, is 70% *smaller*), so the model eliminates 70% of the baseline error.

b) Show that for a linear model with intercept, fitted by least squares **on the same data** on which R^2 is computed, we always have $R^2 \in [0, 1]$.

Hint

The baseline model from a) is itself a member of the linear-model hypothesis space. What does least-squares fitting then imply for $\text{SSE}_{\text{model}}$ compared to $\text{SSE}_{\text{const}}$?

Solution

From a), the featureless baseline is the constant model $f(\mathbf{x}) = \bar{y}$ with $\text{SSE} = \text{SSE}_{\text{const}}$. This intercept-only model $f(\mathbf{x}) = \theta_0$ is contained in the linear-model hypothesis space (set all slope coefficients to 0), and its least-squares fit is indeed $\theta_0 = \bar{y}$, since $\bar{y}$ minimizes

$\sum_{i=1}^{n}(y^{(i)} - c)^2$ over constants c.

Least squares chooses the parameters minimizing the SSE over the **whole** hypothesis space, so the fitted model's SSE cannot exceed that of any particular member – in particular the intercept-only one:

$$0 \leq \mathrm{SSE}_{\mathrm{model}} \leq \mathrm{SSE}_{\mathrm{const}}.$$

Dividing by $\mathrm{SSE}_{\mathrm{const}} > 0$ yields $0 \leq \frac{\mathrm{SSE}_{\mathrm{model}}}{\mathrm{SSE}_{\mathrm{const}}} \leq 1$, hence

$$R^2 = 1 - \frac{\mathrm{SSE}_{\mathrm{model}}}{\mathrm{SSE}_{\mathrm{const}}} \in [0, 1].$$

$R^2 = 1$ corresponds to a perfect fit ($\mathrm{SSE}_{\mathrm{model}} = 0$), and $R^2 = 0$ to a model no better than predicting the mean $\bar{y}$.

c) Now the model is fitted on training data, but R^2 is computed on separate **test** data. Show that R^2 can become negative, state the condition for this, and interpret what a negative R^2 means.

Solution

On test data the argument from b) no longer applies: the model was fitted to minimize the SSE on the *training* data, not on the test data, so there is no guarantee that $\mathrm{SSE}_{\mathrm{model}} \leq \mathrm{SSE}_{\mathrm{const}}$ (with $\bar{y}$ and both SSEs now evaluated on the test set). If the model generalizes poorly, it can predict *worse* than the constant mean baseline, i.e.

$$\mathrm{SSE}_{\mathrm{model}} > \mathrm{SSE}_{\mathrm{const}} \quad \Longrightarrow \quad \frac{\mathrm{SSE}_{\mathrm{model}}}{\mathrm{SSE}_{\mathrm{const}}} > 1 \quad \Longrightarrow \quad R^2 < 0.$$

A negative R^2 therefore means the model performs **worse on the test data than simply always predicting the mean target** – a clear symptom of poor generalization (e.g. overfitting). Unlike on training data, R^2 on test data is not bounded below by 0.